

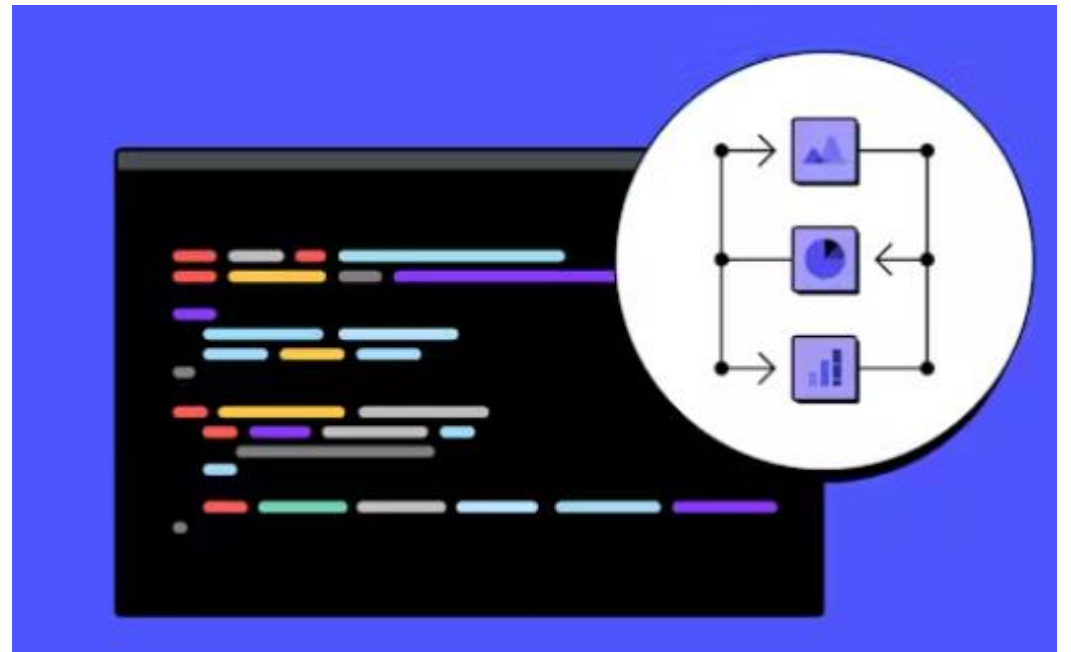
# Data Extraction



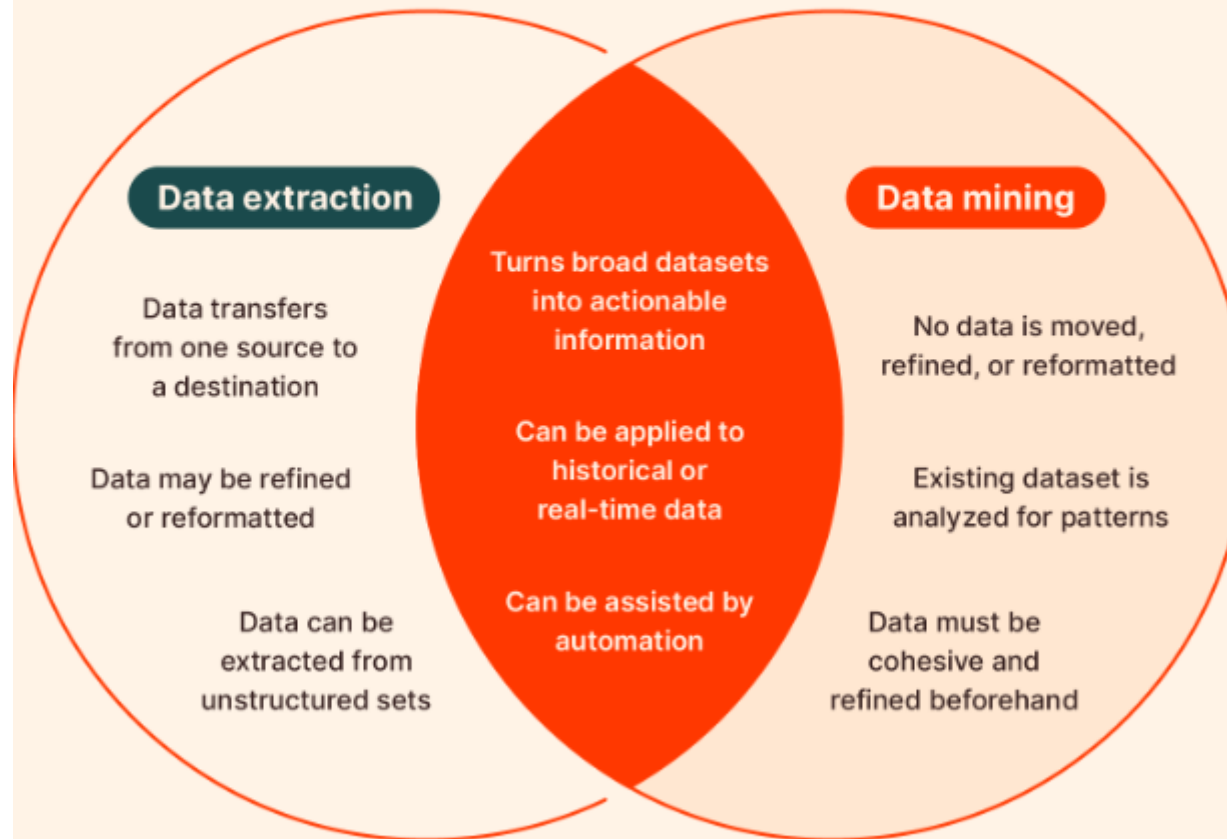
Tim Ajar Business Intelligence

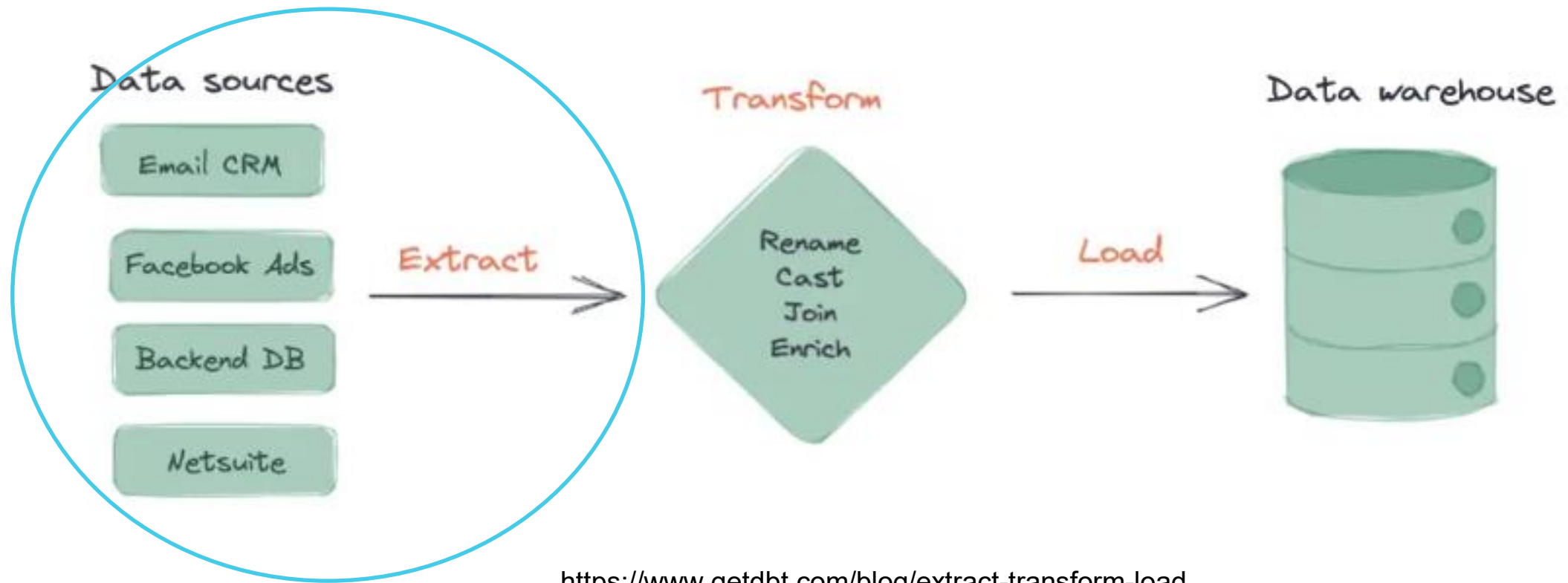
# CAPAIAN PEMBELAJARAN

Mahasiswa mampu memahami konsep Data Extraction



# Data extraction vs. data mining





<https://www.getdbt.com/blog/extract-transform-load>



# TOPIK

1. Prinsip Proses Ekstraksi
2. ETL *approach* dan Arsitekturnya
3. Metode/ Teknik Ekstraksi
4. Mengekstraksi berbagai sumber data

# PENDAHULUAN

**Data Extraction** adalah tahap awal dalam proses **ETL (Extract, Transform, Load)** yang berfungsi untuk mengambil data dari berbagai sumber sebelum dilakukan transformasi dan pemuatan ke dalam data warehouse.

**Tujuan utama ekstraksi** adalah memastikan data yang relevan dapat diambil dengan benar, lengkap, dan efisien dari sistem operasional atau sumber eksternal.

*Data extraction refers to the process of collecting information from various sources for transformation, storage, or feeding it to another system for further analysis.*

## Sumber Data:

Data dapat berasal dari berbagai sumber, di antaranya:

- **Basis data operasional** (OLTP: MySQL, PostgreSQL, Oracle, SQL Server).
- **File semi-terstruktur dan tidak terstruktur** (CSV, Excel, XML, JSON, log file).
- **Aplikasi bisnis** (ERP, CRM, HRM).
- **Web services dan API** (REST, SOAP).
- **Big Data sources** (Hadoop, Spark, NoSQL Database).

# PRINSIP PROSES EKSTRAKSI

- **Waktu dilakukan ekstraksi data** : pada saat melakukan ekstraksi data, sangat penting untuk tidak mengganggu sumber data (*data source*) terlalu banyak.
- **Setelah diekstrak, segera dilakukan transformasi dan dimuat ke datawarehouse, tanpa disimpan sementara di database lain** : data yang dimuat haruslah data yang baik dan bersih (tidak ada karakter data yang tidak terbaca, tidak ada data yang tidak sesuai).
- **Tidak ada *leakage*(kebocoran data)** : Data yang dimuat di data warehouse sama persis seperti data yang ada di *data source*, tidak ada data yang hilang
- ***Recoverability*** : jika terjadi kegagalan, dapat di-*recover* tanpa menghilangkan atau merusak data

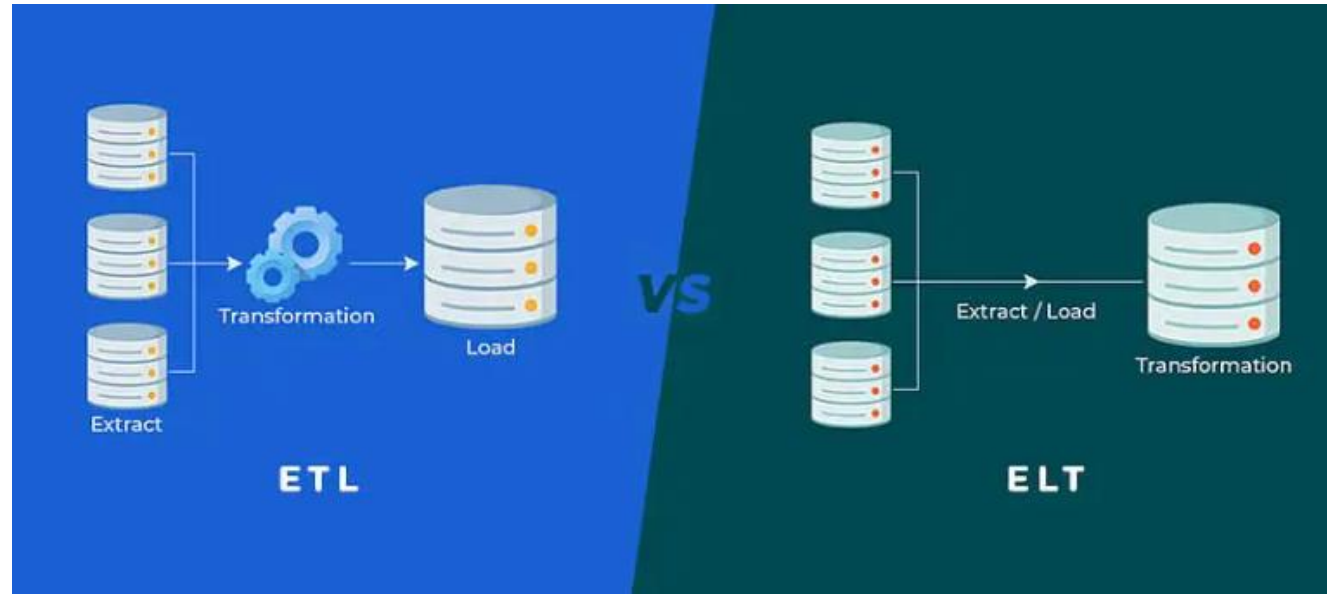




# *ETL APPROACH DAN ARSITEKTURNYA*

---

# BEBERAPA PENDEKATAN ETL



- ETL (Extract, Transform, Load)**

Adalah proses di mana data diekstrak dari sumber, kemudian ditransformasi di staging area atau server ETL, dan setelah itu dimuat ke dalam data warehouse.

- ELT (Extract, Load, Transform)**

Adalah proses di mana data diekstrak dari sumber, langsung dimuat ke data warehouse atau data lake, kemudian transformasi dilakukan di dalam data warehouse itu sendiri.

# PENDEKATAN 1: ETL

- Pendekatan tradisional, menarik data dari data sumber, meletakkannya di staging area, dilakukan transformasi kemudian dimuat ke data warehouse; Atau
- Menarik data dari data sumber, melakukan transformasi di memori (tanpa proses staging) dan langsung dimuat di data warehouse

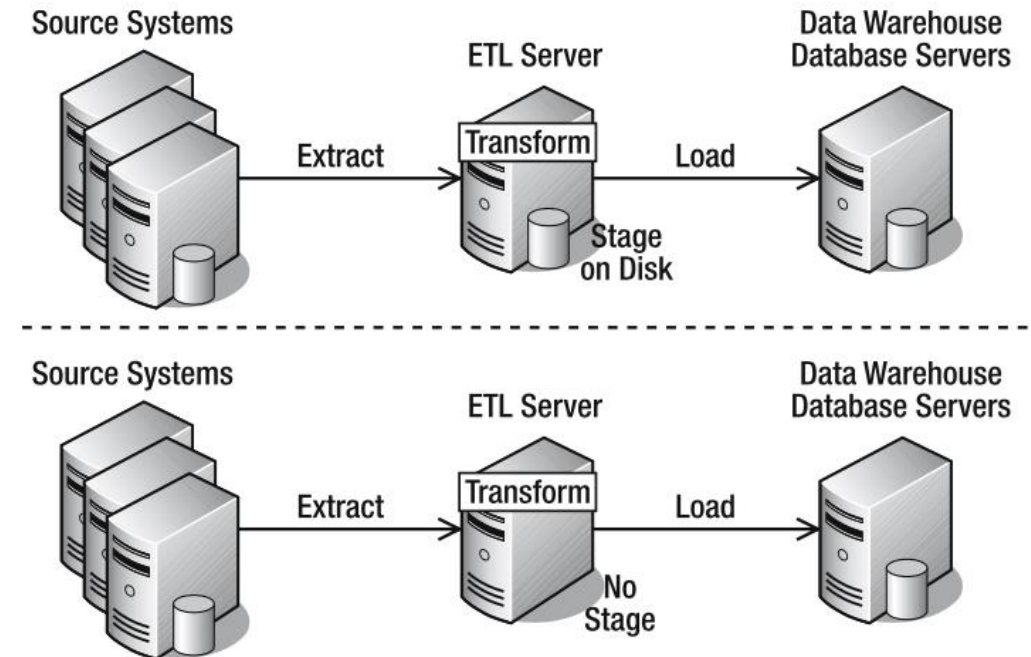
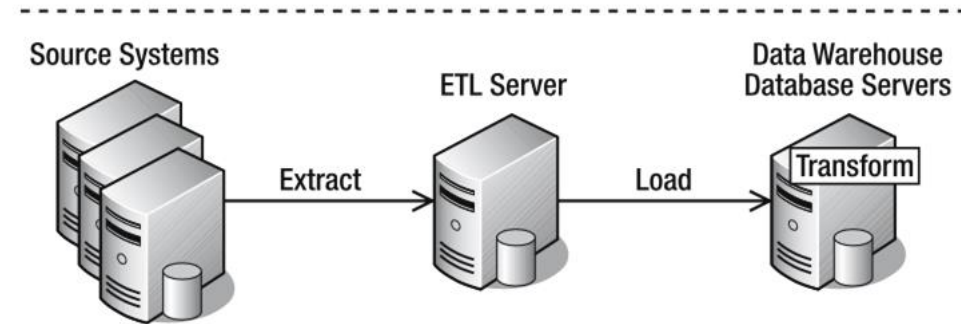


Figure 7-1. To stage on disks or do transformation in memory

# PENDEKATAN 1 : ELT

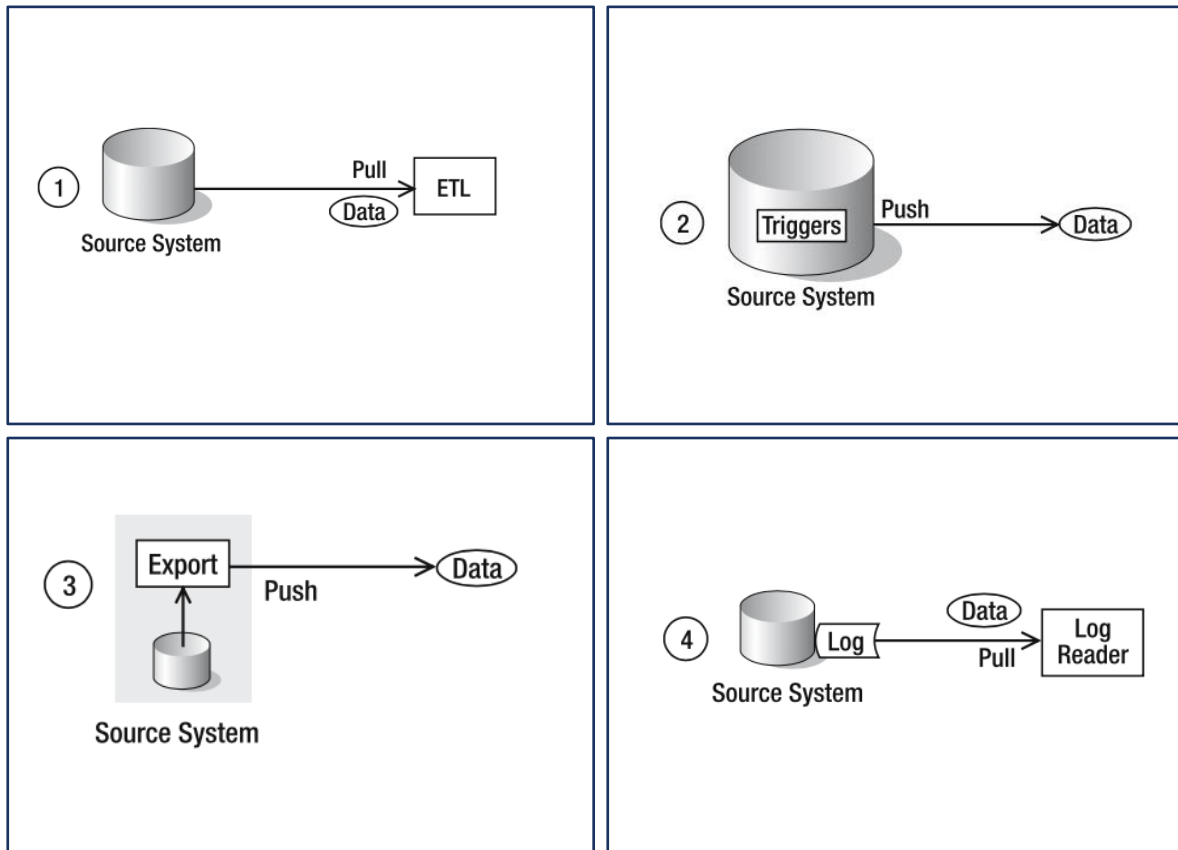
- Pada ELT, proses transformasi dilakukan setelah data dimuat ke data warehouse.



# PERBEDAAN ETL DAN ELT

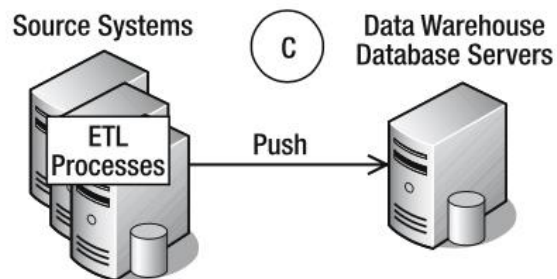
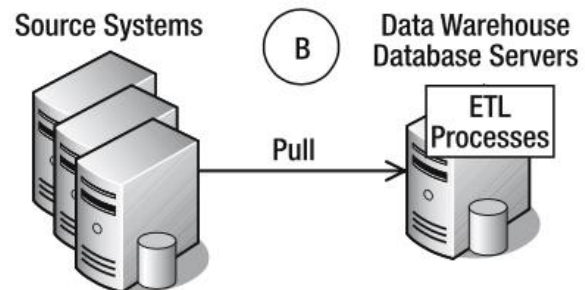
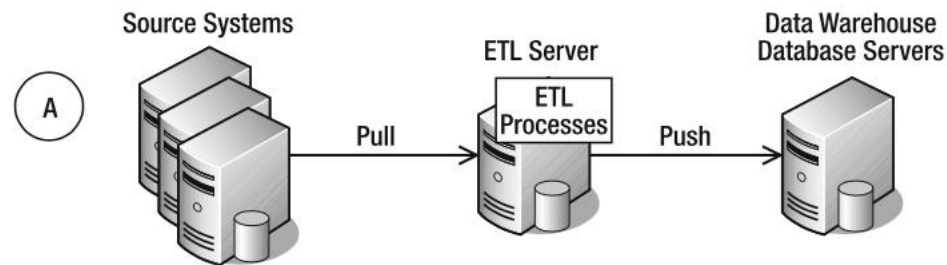
| Aspek                   | ETL                                                                                   | ELT                                                                                                   |
|-------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Lokasi Transformasi     | Dilakukan di <i>staging area</i> atau server ETL sebelum data masuk ke data warehouse | Dilakukan langsung di dalam data warehouse                                                            |
| Arsitektur              | Cocok untuk data warehouse tradisional (schema-based, RDBMS)                          | Cocok untuk modern data warehouse atau data lake berbasis cloud (misal BigQuery, Snowflake, Redshift) |
| Kecepatan               | Bisa lebih lambat pada volume data besar, karena transformasi dilakukan sebelum load  | Lebih cepat untuk big data karena memanfaatkan kekuatan komputasi data warehouse                      |
| Skalabilitas            | Kurang fleksibel saat menangani data tidak terstruktur                                | Lebih mudah menskalakan, termasuk data semi-terstruktur dan tidak terstruktur                         |
| Kebutuhan Infrastruktur | Membutuhkan <i>ETL server</i> khusus untuk transformasi                               | Mengandalkan <i>data warehouse engine</i> untuk transformasi                                          |
| Use Case                | Cocok untuk organisasi dengan sistem OLTP tradisional dan data relatif kecil/menengah | Cocok untuk big data analytics, machine learning, dan cloud-native data processing                    |

# BERDASARKAN SIAPA YANG MENGELUARKAN DATA



1. Dengan menggunakan kueri, proses ETL menarik data secara regular dari sistem database sumber secara regular.
2. Dengan menggunakan trigger.
3. Dengan menggunakan proses terjadwal. Bedanya dengan cara 1, program terjadwal berjalan internal di server data sumber
4. Dengan menggunakan log reader. log reader membaca file log untuk mengidentifikasi perubahan data, menarik data yang berubah dan menyimpannya di tempat lain.

# BERDASARKAN DI MANA DATA DIPROSES



- A. ETL diproses terpisah di ETL server yang ada di antara data sumber dan data warehouse.
- B. ETL dilakukan di data warehouse server
- C. ETL dilakukan di server data sumber

# MASALAH YANG MUNGKIN MUNCUL PADA EKSTRAKSI DATA

| <b>Masalah yang mungkin muncul</b>                                  | <b>Skenario</b>                                                                                                                                         | <b>Cara umum untuk mengatasinya</b>                                                             |
|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Data hanya tersedia di waktu tertentu                               | Jadwal batch yang panjang berjalan semalaman diikuti dengan pencadangan database yang besar. Mengekstraksi data pada siang hari akan memperlambat OLTP. | Negosiasikan waktu dengan OLTP DBA. Pecahkan hasil ekstraksi menjadi potongan-potongan kecil.   |
| Database OLTP tidak tersedia pada hari-hari tertentu.               | Pada hari pertama setiap bulan, OLTP menjalankan batch akhir bulan sepanjang hari.                                                                      | Lewati ekstraksi pada hari itu. Beri tahu pengguna alasan pembatasan ini.                       |
| DBA OLTP khawatir bahwa ETL kami dapat “mengacaukan” sistem mereka. | Tim proyek terdiri dari konsultan eksternal yang baru mempelajari internal OLTP.                                                                        | Minta DBA untuk memberikan akses “read-only”. Minta DBA OLTP untuk mengeksport data untuk Anda. |



# MASALAH YANG MUNGKIN MUNCUL PADA EKSTRAKSI DATA

| <b>Masalah yang mungkin muncul</b>                                                        | <b>Skenario</b>                                             | <b>Cara umum untuk mengatasinya</b>                                                                                                        |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Ada kekhawatiran bahwa ETL akan membebani/membeani database OLTP.                         | Tidak ada “masa tenang” karena OLTP adalah database global. | Baca dari database OLTP sekunder. Gunakan ekstrak yang lebih jarang untuk tabel besar, seperti hanya pada hari Minggu.                     |
| DBA OLTP khawatir bahwa mereka tidak dapat menghentikan proses ETL dalam keadaan darurat. | Ada situasi darurat di OLTP yang memerlukan restart server. | Beritahu DBA bahwa mereka dapat memberhentikan Anda kapan saja (beri tahu caranya) namun jalankan kembali setelahnya (pemanggilan manual). |
| Mereka tidak dapat memulai proses ETL.                                                    | Ada kelebihan cadangan OLTP.                                | Program ETL untuk dijalankan kembali secara otomatis nanti.                                                                                |



# MENGEKSTRAKSI DARI BERBAGAI SUMBER DATA

---

# DARI DATABASE RELASIONAL

- Ada beberapa metode yang bisa digunakan
  1. seluruh tabel setiap waktu (*whole table every time*)
  2. Ekstraksi incremental (*incremental extract*)
  3. Range tetap (*Fixed range*)
  4. *Push approach*

# DARI DATABASE RELASIONAL

## 1. Whole table every time

jika ukuran tabel kecil (berisi hingga 10.000 baris data) atau jika tidak ada timestamp atau identitas kolom yang menunjukkan baris mana yang diperbarui

**Definisi:** Seluruh isi tabel diambil setiap kali proses ekstraksi dilakukan, tanpa memperhatikan apakah ada perubahan data atau tidak.

**Contoh:** Digunakan pada tabel dimensi kecil seperti daftar kategori produk.

| Table | Code | Description         |
|-------|------|---------------------|
| PMT   | 1    | Direct debit        |
| PMT   | 2    | Monthly invoice     |
| PMT   | 3    | Annual in advance   |
| STS   | AC   | Active              |
| STS   | SU   | Suspended           |
| STS   | BO   | Balance outstanding |
| SUB   | S    | Subscribed          |
| SUB   | U    | Unsubscribed        |
| ...   |      |                     |

# DARI DATABASE RELASIONAL

## 2. Incremental extract

Definisi: Hanya data yang baru ditambahkan atau diubah sejak proses ekstraksi terakhir yang diambil.

Teknik umum:

- Timestamp (tanggal/waktu update).
- Change Data Capture (CDC) melalui log transaksi.
- Flag status perubahan (misalnya kolom is\_updated).

Yang perlu dipastikan adalah apakah timestamp dapat diandalkan (apakah Ketika data berubah, timestamp juga berubah, apakah timestamp lengkap tidak ada nilai null, apakah last updated tidak lebih kecil dari created)

**Contoh:** Transaksi penjualan harian pada sistem ritel.

**Table 7-4.** *Jade Order Header Table*

| Order ID | Order Date | Some Columns | Order Status | Created                | Last Updated           |
|----------|------------|--------------|--------------|------------------------|------------------------|
| 45433    | 10/10/2007 | Some Data    | Dispatched   | 10/11/2007<br>10:05:44 | 10/12/2007<br>11:23:41 |
| 45434    | 10/15/2007 | Some Data    | Open         | 10/16/2007<br>14:10:00 | 10/17/2007<br>15:29:02 |
| 45435    | 10/16/2007 | Some Data    | Canceled     | 10/16/2007<br>11:23:55 | 10/17/2007<br>16:19:03 |
| ...      |            |              |              |                        |                        |

# DARI DATABASE RELASIONAL

## 3. Fixed Range

**Definisi:** Ekstraksi dilakukan berdasarkan **rentang nilai tertentu**, misalnya rentang tanggal atau nomor transaksi

**Contoh:** Mengambil data transaksi penjualan hanya dari tanggal 1–30 September 2025.

# DARI DATABASE RELASIONAL

## 4. Push Approach

Jika ada perubahan data pada data sumber, maka perlu ada perubahan juga dengan data warehouse.

**Definisi:** Data **didorong (push)** langsung oleh sistem sumber ke sistem tujuan (ETL server atau data warehouse), bukan ditarik (pull) oleh proses ETL.

**Contoh:** Sistem POS (Point of Sale) mengirim data transaksi ke data warehouse secara langsung melalui API atau message queue (misalnya Kafka).

## Pemilihan metode ekstraksi tergantung pada:

- **Ukuran data** (besar/kecil).
- **Kebutuhan pembaruan** (batch harian, near real-time).
- **Ketersediaan infrastruktur** (database log, API, message broker).

Umumnya, organisasi menggabungkan beberapa metode: **whole table** untuk tabel dimensi kecil, **incremental** untuk data transaksi, dan **push** untuk kebutuhan real-time.



# Skenario Kombinasi Metode Ekstraksi Data Ritel

Sebuah perusahaan ritel besar membangun **data warehouse** untuk analisis penjualan, persediaan, dan perilaku pelanggan. Sistem operasional yang digunakan mencakup:

- **POS (Point of Sale)** untuk transaksi harian.
- **ERP** untuk manajemen persediaan dan produk.
- **CRM** untuk data pelanggan.

Untuk menjaga efisiensi, digunakan **kombinasi metode ekstraksi** sebagai berikut:

### I. Whole Table Every Time (Dimensi Kecil)

- **Tabel target:** dim\_kategori, dim\_supplier.
- **Alasan:**
  - Ukuran tabel relatif kecil (ratusan hingga ribuan baris).
  - Perubahan jarang terjadi.
  - Lebih sederhana mengambil seluruh isi tabel setiap kali ETL dijalankan.
- **Frekuensi:** Harian saat batch processing (misalnya pukul 23.00).

## 2. Incremental Extract (Transaksi Harian)

- **Tabel target:** fact\_penjualan, fact\_pembayaran.
- **Alasan:**
  - Volume transaksi besar (ribuan hingga jutaan per hari).
  - Tidak efisien mengambil seluruh tabel.
  - Hanya data transaksi baru/diubah sejak ETL terakhir yang diambil.
- **Teknik:** Menggunakan kolom last\_update atau log transaksi database (CDC).
- **Frekuensi:** Setiap malam untuk batch, atau setiap 1 jam untuk near real-time.



### 3. Fixed Range Extraction (Data Historis)

- **Tabel target:** fact\_penjualan\_historis.
- **Alasan:**
  - Saat membangun data warehouse pertama kali, diperlukan data historis dari 5 tahun terakhir.
  - Dilakukan ekstraksi berdasarkan rentang tanggal (misalnya 2018–2022).
- **Teknik:** Query dengan WHERE tanggal BETWEEN '2018-01-01' AND '2022-12-31'.
- **Frekuensi:** Satu kali saat initial load, bisa diulang untuk reprocessing bila ada koreksi.

#### 4. Push Approach (Data Real-Time)

- **Tabel target:** fact\_log\_transaksi\_online.
- **Alasan:**
  - Perusahaan memiliki e-commerce yang membutuhkan pemantauan transaksi real-time (misalnya untuk mendeteksi fraud atau memantau stok online).
  - Sistem e-commerce mendorong data transaksi ke data warehouse melalui **API** atau **message queue (Apache Kafka/ActiveMQ)**.
- **Frekuensi:** Real-time (langsung saat transaksi terjadi).

# Kesimpulan

## Skenario Ekstraksi Data:

- **Dimensi statis (kategori, supplier)** → *whole table extract*.
- **Transaksi harian (penjualan, pembayaran)** → *incremental extract*.
- **Data historis (arsip penjualan lama)** → *fixed range*.
- **Transaksi online e-commerce** → *push approach*.

Optimal dengan mengombinasikan metode ekstraksi:

- **Efisiensi kinerja** tetap terjaga (tidak membebani sistem sumber).
- **Kelengkapan data** tetap terjamin (historis + transaksi berjalan).
- **Fleksibilitas integrasi** tercapai (batch + near real-time + real-time).



THANK YOU

